MIT Art Design and Technology University

MIT School of Computing, Pune

Title: Inventory Management System

Group Members:

Roll No	Name
26	Isha Chaniyara
27	Ish Chaniyara
29	Bhakti Kulkarni
30	Ajinkya More

Prof. Deepak Naik

Introduction

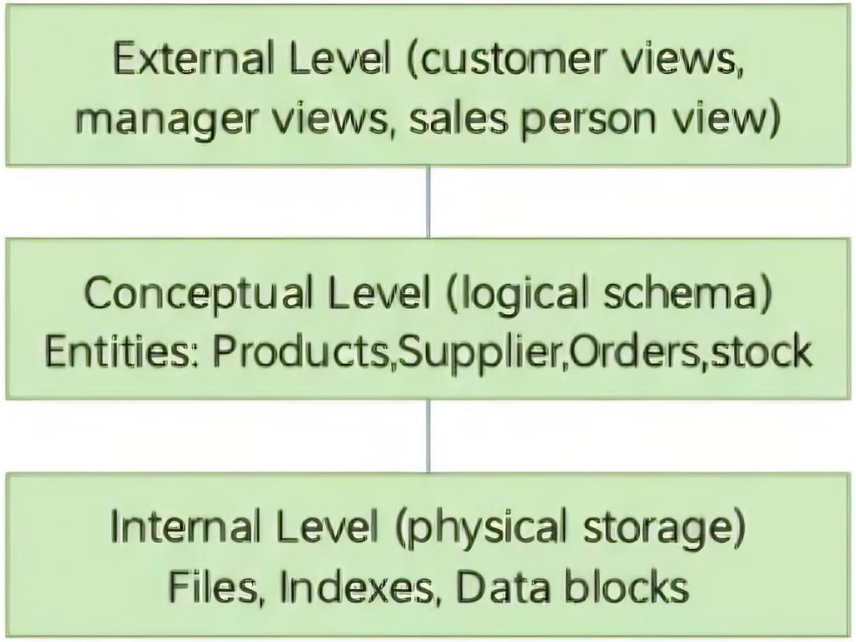
- **Definition:** An Inventory Management System (IMS) is a software/database solution used to track, manage, and control stock levels.
- **Purpose:** Ensures efficient handling of products, suppliers, customers, and orders.

- Provides real-time updates for better decisions.
- DBMS ensures accuracy, security, and easy access to data.
- Improves efficiency and reliability in inventory control.

- **Outcome:** IMS with DBMS improves efficiency, accuracy, and scalability in managing inventory.

Database Architecture

- External Level (user views like manager, supplier, customer).
- Conceptual level (tables & relations)
- Internal Level (physical storage)



Database Architecture

- **1. External Schema (User Views)**
- This is the **top level**, closest to users.
- It shows **different views for different users** depending on their role.
- In IMS:
  - **Inventory Manager View** → Can see product stock levels, warehouse status, low-stock alerts.
  - **Procurement Officer View** → Can see supplier details, purchase orders, pending deliveries.
  - **Sales Manager / Customer View** → Can see customer orders, invoices, and sales reports.
  - **Admin View** → Full access to all data (Products, Suppliers, Customers, Orders, Stock).

- The Inventory Manager's screen only shows product stock details, not customer or supplier details.

Database Architecture

- **2. Conceptual Schema (Logical Schema)**
- Shows the **entire database design logically**.
- Example tables/entities:
  - **Products** → ProductID, Name, Category, Price, StockLevel
  - **Suppliers** → SupplierID, Name, ContactInfo
  - **Orders** → OrderID, SupplierID, Date, TotalAmount
  - **Inventory** → ProductID, Warehouse, Quantity

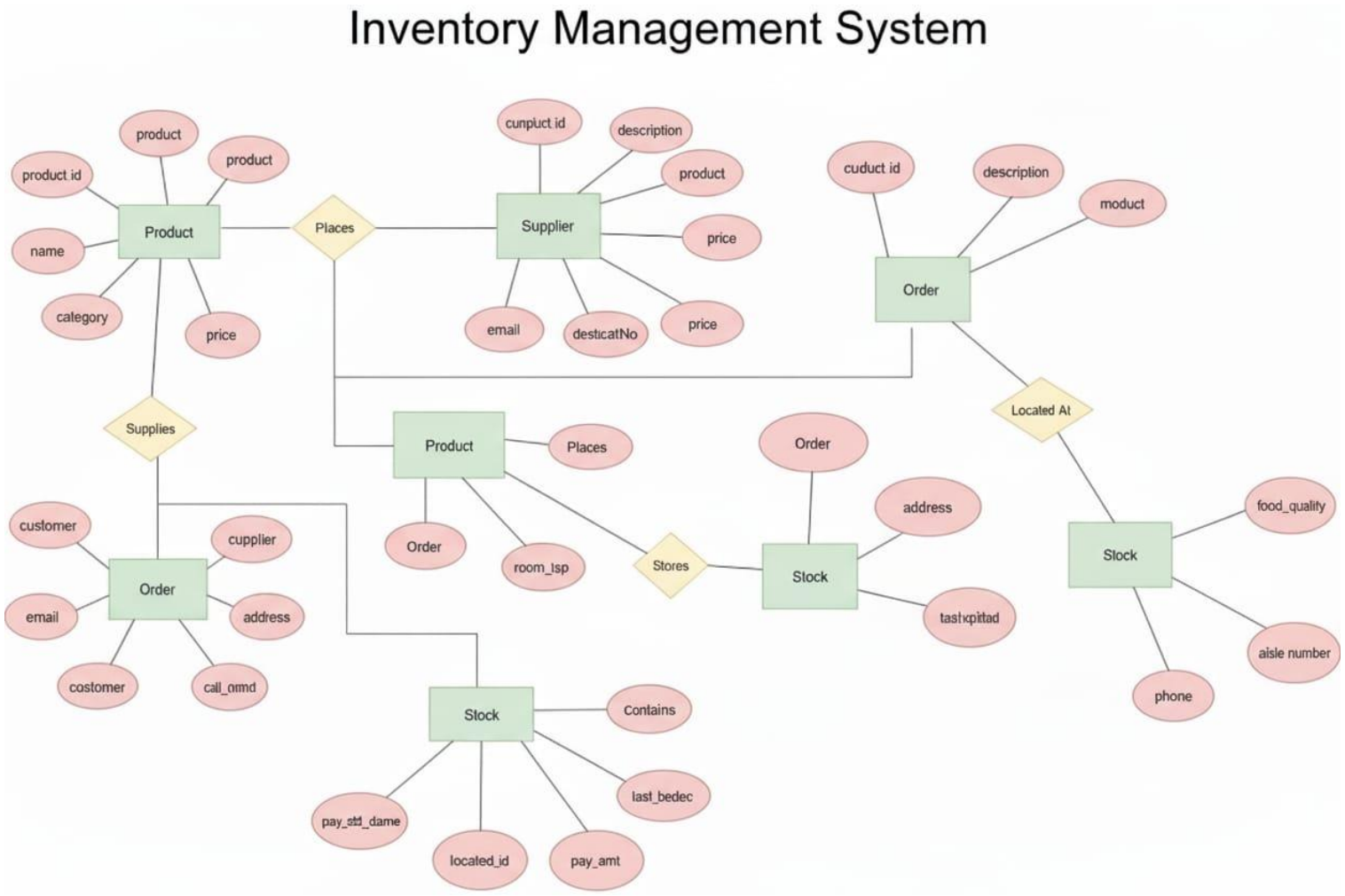
- At this level we know "A Supplier supplies Products" and "Orders include Products," but we don't see *how* they are stored physically.

Database Architecture

- **3. Internal Schema (Physical Schema)**
- Describes **how data is actually stored** in the database system.
- Example in IMS:
  - Product details stored in data files on disk.
  - Index built on **ProductID** for fast product lookup.
  - Orders stored in separate partitions by date.
  - Backup storage ensures recovery of inventory records.

- When you search for a product in IMS, the DBMS fetches it quickly using indexes — but users don’t see this process.

ER Diagram


Inventory Management System
product
product
product id
Product
name
category
price
Places
cunpuct id
description
product
Supplier
price
email
destıcatNo
price
cuduct id
description
moduct
Order
Supplies
Product
Places
Order
room_lsp
Stores
Stock
address
tastıxpittad
Located At
food_quality
Slock
aisle number
phone
customer
cupplier
Order
email
address
costomer
call_ommd
Stock
Contains
last_bedec
pay_stıl_dame
located_id
pay_amt

Tables

Product	Supplier	Customer	Orders	Stock
Product_ID (PK)	Supplier_ID (PK)	Customer_ID (PK)	Order_ID (PK)	Stock_ID (PK)
Name	Name	Name	Customer_ID (FK) → Customer.Customer_ID	Product_ID (FK) → Product.ID
Category	Email	Email	Supplier_ID (FK) → Supplier.Supplier_ID	Warehouse_Location
Price	Contact_No	Phone	Product_ID (FK) → Product.Product_ID	Quantity
Stock_Level	Address	Address	Order_Date	Aisle_Number
-	-	-	Quantity	Food_Quality
-	-	-	Total_Amount	Phone

Query 1 – Restocking Products

- Operation: Find products that need immediate restock.

- Relational Algebra:

$\sigma(\text{StockLevel} < 10)\ (\text{Products})$

- SQL:

```sql
SELECT ProductID, Name, StockLevel
FROM Products
WHERE StockLevel < 10;
```

Output

ProductID	Name	StockLevel
P101	Laptop	5
P202	Keyboard	3

Query 2 – Supplier Performance

- Operation: Rank suppliers by supply value.

- Relational Algebra:

π(SupplierID, ProductID, SUM(Quantity)) (Orders $\bowtie$ Products)

- SQL:

```sql
SELECT s.SupplierID, s.Name, o.OrderID, o.TotalAmount
FROM Suppliers s
JOIN Orders o ON s.SupplierID = o.SupplierID;
```

- Ouput

SupplierID	Name	OrderID	TotalAmount
S01	Tech Distributors	O501	5000
S02	Prime Supplies	O502	3000
S01	Tech Distributors	O503	7000

Query 3 – Common Items Between Warehouse

- Operation: Find overlapping items in warehouses A and B.

- Relational Algebra:

(Inventory[Warehouse=A] ∩ Inventory[Warehouse=B])

- SQL

```sql
SELECT ProductID, Quantity
FROM Inventory
WHERE Warehouse = 'A';
```

- Output

ProductID	Quantity
P101	50
P202	30
P303	20

Query 4 – Products with Orders (Inner Join)

- Operation: List ordered products.

- Relational Algebra:

Orders ⋈ Products

- SQL:

```sql
SELECT o.OrderID, p.Name, o.Date
FROM Orders o
JOIN Products p ON o.ProductID = p.ProductID;
```

OUPUT

OrderID	Name	Date
O501	Laptop	2025-08-10
O502	Keyboard	2025-08-12
O503	Mouse	2025-08-13

Query 5 – All Products by Category

- Operation : Inventory value by category

- Relational Algebra:
γ(Category, SUM(Price*StockLevel))(Products)

- SQL Query:

```sql
SELECT ProductID, Name, Category, Price, StockLevel
FROM Products;
```

- OUPUT

ProductID	Name	Category	Price	StockLevel
P101	Laptop	Electronics	50000	5
P202	Keyboard	Electronics	2000	3
P303	Pen	Stationery	10	100

Aggregation & Grouping

- Total stock of each product

```sql
SELECT SUM(QuantityAvailable) AS TotalStock
FROM Inventory;
```

- Total orders per customer

```sql
SELECT COUNT(OrderID) AS TotalOrders
FROM Orders;
```

- Average order quantity of products

```sql
SELECT AVG(Quantity) AS AvgOrderQty
FROM Orders;
```

Joins

- INNER JOIN: Get all orders with product details

```sql
SELECT Orders.OrderID, Customers.Name AS CustomerName,
       Products.Name AS ProductName, Orders.Quantity
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID
INNER JOIN Products ON Orders.ProductID = Products.ProductID;
```

- LEFT JOIN: Show all products even if not ordered

```sql
SELECT Products.Name AS ProductName, Orders.OrderID
FROM Products
LEFT JOIN Orders ON Products.ProductID = Orders.ProductID;
```

- RIGHT JOIN: Show all orders even if product details missing

```sql
SELECT Products.Name AS ProductName, Orders.OrderID
FROM Products
RIGHT JOIN Orders ON Products.ProductID = Orders.ProductID;
```

Set Operations

- **UNION**

Combines results from two queries into **one list (no duplicates)**.

```sql
SELECT Name FROM Products
UNION
SELECT Name FROM Suppliers;
```

- **INTERSECT**

Returns only the **common values** between two queries.

```sql
SELECT Name FROM Customers
INTERSECT
SELECT Name FROM Suppliers;
```

- **EXCEPT (or MINUS)**

Returns values from the **first query that are not in the second**.

```sql
SELECT ProductID FROM Products
EXCEPT
SELECT ProductID FROM Orders;
```